

Black lives matter

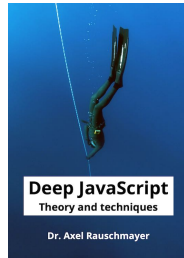
2ality – JavaScript and more

[About](#) | [Donate](#) | [Subscribe](#) | [Search](#) | [Archive](#) | [Books](#)

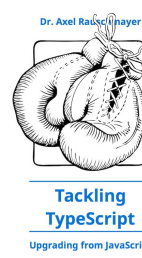
Dr. Axel
Rauschmayer
[Homepage](#) |
[Twitter](#)



Book, exercises,
quizzes
(free to read
online)



Book (50% free
online)



Book (first part
free online)



Newsletter (free)

ES2020: `import()` – dynamically importing ES modules

[2017-01-09] [dev](#), [javascript](#), [es2020](#), [jsmodules](#)

(Ad, please don't block)



Adobe Creative Cloud for Teams starting at \$33.99 per month.

ADS VIA CARBON

The ECMAScript proposal “`import()`” by Domenic Denicola is currently at stage 3. It enables dynamic loading of ECMAScript modules and is explained in this blog post.

1 ECMAScript modules are static

ECMAScript modules are completely static: you must specify what you import and export at compile time and can't react to changes at runtime. That has several advantages, especially w.r.t. tooling, [which are explained in “Exploring ES6”](#).

The static structure of imports is enforced syntactically in two ways. Consider the following example:

```
import * as someModule from './dir/someModule.js';
```

First, this import declaration can only appear at the top level of a module. That prevents you from importing modules inside an `if` statement or inside an event handler.

Second, the *module specifier* `'./dir/someModule.js'` is fixed; you can't compute it at runtime (via a function call etc.).

2 The proposal enables dynamic module imports

The proposed operator for loading modules dynamically works as follows:

```
const moduleSpecifier = './dir/someModule.js';
import(moduleSpecifier)
  .then(someModule => someModule.foo());
```

The operator is used like a function:

- The parameter is a string with a module specifier that has the same format as the module specifiers used for `import` declarations. In contrast to the latter, the parameter can be any expression whose result can be coerced to a string.
- The result of the “function call” is a Promise. Once the module is completely loaded, the Promise is fulfilled with it.

Even though it works much like a function, `import()` is an operator: In order to resolve module specifiers relatively to the current module, it needs to know from which module it is invoked. Normal functions have no straightforward way of finding that out.

3 Use cases

3.1 Loading code on demand

Some functionality of web apps doesn't have to be present when they start, it can be loaded on demand. Then `import()` helps, because you can put such functionality into modules. For example:

```
button.addEventListener('click', event => {
  import('./dialogBox.js')
    .then(dialogBox => {
```

```
        dialogBox.open();  
    })  
    .catch(error => {  
        /* Error handling */  
    })  
});
```

3.2 Conditional loading of modules

Sometimes you may want to load a module depending on whether a condition is true. For example, to load a polyfill on legacy platforms. That looks as follows.

```
if (isLegacyPlatform()) {  
    import(...)  
    .then(...);  
}
```

3.3 Computed module specifiers

For applications such as internationalization, it helps if you can dynamically compute module specifiers:

```
import(`messages_${getLocale()}.js`)  
    .then(...);
```

4 Tips

4.1 Accessing exports via destructuring

Destructuring helps with accessing a module's exports:

```
import('./myModule.js')  
    .then(({export1, export2}) => {  
        ...  
    });
```

4.2 Accessing default exports

For default exports, you need to know that `default` is a keyword. Using it as a property name via the dot notation is OK:

```
import('./myModule.js')
  .then(myModule => {
    console.log(myModule.default);
  });
```

However, you can't use it as a variable name:

```
import('./myModule.js')
  .then(({default: theDefault}) => {
    console.log(theDefault);
  });
```

4.3 Dynamically loading multiple modules

You can dynamically load multiple modules at the same time via `Promise.all()`:

```
Promise.all([
  import('./module1.js'),
  import('./module2.js'),
  import('./module3.js'),
])
  .then([module1, module2, module3]) => {
    ...
  });
```

4.4 Async functions and `import()`

`import()` returns Promises, which means that you can use it via **async functions** (which are part of ECMAScript 2017) and get nicer syntax:

```
async function main() {
  const myModule = await import('./myModule.js');

  const {export1, export2} = await import('./myModule.js');

  const [module1, module2, module3] =
    await Promise.all([
      import('./module1.js'),
      import('./module2.js'),
      import('./module3.js'),
    ])
```

```
    ] );  
  }  
  main();
```

At the top-level of a module or script, you may find Immediately Invoked Async Arrow Functions useful:

```
(async () => {  
    const myModule = await import('./myModule.js');  
}) ();
```

5 Support for `import()`

- Node.js: Guy Bedford's [node-es-module-loader](#) provides a Node.js executable that supports ES6 module syntax and `import()`.
- webpack v1: [babel-plugin-dynamic-import-webpack](#) is a Babel plugin that transpiles `import()` to `require.ensure()`.
- webpack v2 ([v2.1.0-beta.28](#) and later): supports [code splitting via `import\(\)`](#)

6 Further reading

- Chapter “[Modules](#)” in “[Exploring ES6](#)”
- Chapter “[Promises for asynchronous programming](#)” in “[Exploring ES6](#)”
- Chapter “[Async functions](#)” in “[Exploring ES2016 and ES2017](#)”